

HW06-AI API Testing Report

Student and submission

Field	Value
Student	Nguyen Bao Duy
Student ID	23127179
Class	23KTPM2
Assignment	HW06-AI — API Testing
Public repository	https://github.com/BuhDuy256/automation-testing-hw04
Branch	hw06-api-testing
Self-assessed grade	100/100

Scope and method

The submission tests one operation from each required pool: FR-04 `PUT /api/users/me`, FR-08 `POST /api/checkout`, and FR-15 `POST /api/products`. For each operation, AI generated source-anchored candidates in bounded coverage batches; the student reviewed every AI candidate, corrected or rejected unsuitable cases, selected at least five extensions, approved the executable Postman design, reviewed real Newman evidence, confirmed genuine defects, and authorized external Issue publication. Test oracles use documented state invariants and read-back evidence instead of invented status codes or schemas.

Test accounting

API	AI generated	Human added	Audited	Executable	Executed	Result
FR-04	47	7	47	44	44	35 PASS, 9 FAIL; 8 failures map to two bugs and one is a specification-gap observation
FR-08	57	5	57	56	56	37 PASS, 19 FAIL; all failures map to two bugs
FR-15	59	5	59	59	59	16 authoritative PASS, 28 FAIL, 15 PASS-shaped SPEC-GAP observations
Total	163	17	163	159	159	56 failures; 55 are mapped to confirmed defects

The detailed cases, original AI output, human verdicts, corrections, execution classifications, and summary worksheets are in [test-cases.xlsx](#). Canonical source data is included under [provenance/](#).

FR-04 — Personal profile management

The suite covers editable profile partitions, phone boundaries, authentication, protected email/role behavior, persistence, stateful repeated updates, Unicode and input-handling observations. The canonical executions preserve genuine failures. Invalid phone values persisted, and an ordinary user could persist `role=admin`. These root defects were published as Issues [#13](#) and [#14](#). Artifacts are under [fr04/](#).

FR-08 — Checkout

The suite creates isolated users and cart states, derives the expected total from the server-reported cart, checks order persistence, exercises authentication and malformed inputs, and observes cart state after checkout. The backend trusted client `total_amount` and did not clear the cart; Issues [#15](#) and [#16](#) record the evidence. A client total that coincidentally equals the cart total can pass without proving backend recalculation, so this limitation is explicit. Artifacts are under [fr08/](#).

FR-15 — Product creation

The suite covers name, price, category, body shape, admin authorization, invalid JWT contexts, persistence, duplicate/cleanup observations, and security boundaries. Product creation succeeded without a valid admin context and accepted documented-invalid product data; Issues [#17](#) and [#18](#) record the two root defects. Fifteen assertion-clean results remain SPEC-GAP observations because the official contract does not define the exact behavior needed for a stronger verdict. Artifacts are under [fr15/](#).

Postman and Newman

The collections use environments, variables, data-driven iterations, pre-request scripts, test scripts, `pm.sendRequest` read-back/cleanup helpers, and Newman JSON/HTML reporters. Every collection injects `X-Student-Id: 23127179`; the student-attested console screenshot is [here](#). The complete feature list and evidence mapping are in [postman-features.md](#).

Bugs

Six genuine defects are documented in [bug-report.md](#). Each published Issue has a public URL and an included student-attested screenshot. Exact undocumented error statuses are not claimed; persistence and state observations form the primary bug oracles.

CI/CD

The CI configuration starts the SUT, runs Newman, uploads evidence, and exposes the real exit result. The required green/red sample pair is supported by real commits and GitHub Actions runs. The complete canonical FR-04 workflow intentionally remains red because it retains confirmed SUT-defect failures; the smaller all-pass sample demonstrates CI mechanics and is not presented as proof that the entire canonical suite is green. See [ci-cd-report.md](#).

AI-driven test generator

The reusable Agent Skill is packaged at [generator/agent-skill/SKILL.md](#), and its design is expressed as submission-facing [pseudocode](#). It separates contract extraction, domain partitions, state, security, schema, deduplication, source-grounded oracles, and provenance stamping from the later human-controlled workflow. The student-designed workflow diagram is included as a [PNG](#) with its editable [Draw.io XML source](#).

AI use, critique, and audit

The project-specific 200–300 word critique is [ai-critique.md](#). The complete AI declaration and artifact-level audit are supplied as [Markdown](#) and [PDF](#). The audit distinguishes preserved prompts from historical context fallbacks and never assigns human verdicts without recorded student review.

Limits and honest interpretation

- SEC-04 concerns the display boundary; API read-back of HTML-like input cannot prove safe UI escaping.

- Black-box requests may expose injection vulnerabilities but cannot prove SEC-05 parameterized-query implementation.
- A coincidental PASS is not evidence of the intended internal backend mechanism.
- The three-API group non-duplication check needs external student confirmation because no group allocation source is present.
- The student-drawn generator diagram and YouTube generator demonstration are both linked from [README.md](#).

Submission map

The complete requirement classification and official 100-point self-assessment are in [submission-checklist.md](#). Real Git workflow evidence is in [git-commit-log.txt](#). All local links in this submission are checked to remain inside this directory, and no runtime database or active secret is included.